

可视计算与交互概论 Lab 1

Dithering

Uniform Random Dithering

任务目标是对图像上每个像素做一个 uniform random dithering 操作，给每个像素加上 $[-0.5, 0.5]$ 的随机噪声。我们选择使用 `std::mt19937` 算法生成随机噪声。由于该算法只能生成整数噪声，我们可以将生成范围调整为 $[-500, 500]$ ，再将结果除 10, 就可以生成小数均匀噪声了。对于每个像素的 R,G,B 通道都加上同一个噪声，最后二值化，就得到了需要的结果。



Figure 1: Uniform Random Dithering

Blue Noise Random Dithering

由于蓝噪声的纹理已经给出，直接逐像素相加，得到一个错误的结果。



Figure 2: Blue Noise Random Dithering(wrong)

主要是因为蓝噪声的像素范围是 $[0, 1]$, 相当于在图片的期望上加了 0.5, 导致图片严重偏亮。我们减去对每个像素的每个通道都减去 0.5



Figure 3: Blue Noise Random Dithering(correct)

Ordered Dithering

按照课件的像素点亮顺序，先将每个点的灰度 $\times 10$ ，然后将得到的数值只保留整数部分，就得到了对应的点亮位置。按照要求点亮即可。为了降低代码复杂度，我们对代码的写法做了一定优化，具体见代码。



Figure 4: Ordered Dithering

Error Diffuse Dithering

实现标准的 Floyd-Steinberg Error Diffusion。先将 output 设置为 input，然后由上到下、由左到右依次二值化，将误差乘以对应系数直接加到对应的位置上。得到结果



Figure 5: Error Diffuse Dithering

Task 2: Image Filtering ($2 \times 1' = 2'$)

Gaussian Blur

先实现了一下卷积操作（实际是 correlation 而不是 convolution，但和 pytorch 里面定义的一样）

实现方法：从左上角向右下角枚举，将枚举的点作为起始点，用卷积核做卷积操作，之后直接赋值到输出矩阵。对于卷积核所有元素之和不为 0 的，要做一个归一化操作，避免卷积后图像和原图亮度有差异。

对于高斯模糊核，我们选择最简单的 $\frac{1}{16} \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$ ，直接卷积即可。

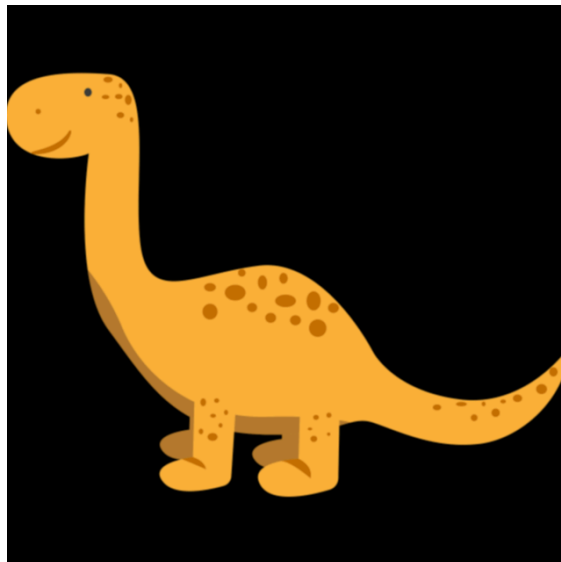


Figure 6: 模糊，大小为原图的 30%

Edge Detection

首先定义两个边缘检测核，用作 x 和 y 方向的边缘检测。其定义与 ppt 一致。

对原图分别使用两个卷积核进行卷积，将生成的两个边缘检测结果 A, B 逐像素做 $C_{i,j} = \sqrt{A_{i,j}^2 + B_{i,j}^2}$, C 就是边缘检测得到的图像。

然而需要注意的是，ImageRGB 不允许负数的出现，所以导致结果只有一个方向的边缘。为了方便，我们修改 Convolution 函数，让其获得卷积结果的时候直接做绝对值。虽然这个不是很符合要求，但确实能正常运行。

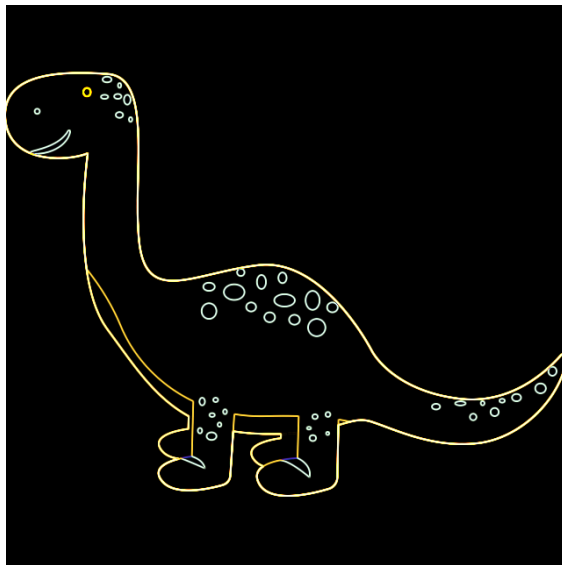


Figure 7: 边缘检测，大小为原图的 30%

Task 4: Line Drawing (1')

前期尝试

这个题目是我第一个写的题目，用这个题目对库的基本用法做一个尝试。

首先试着写一个最朴素的算法，看看各个库怎么用。

```
int x0 = p0.x, y0 = p0.y, x1 = p1.x, y1 = p1.y;

for(int i = x0; i <= x1; i++)
{
    float ny = (y1 - y0) / (float)(x1 - x0) * (i - x0) + y0;
    canvas.At(i, (int)round(ny)) = color;
}
```

实现

在清楚整体框架后，我们可以开始实现一个 Bresenham 算法。

课上讲解的 Bresenham 算法不能很好处理在 2,3,4 象限和 $k > 1$ 的情况，这里做以下修改：

1. 使用 err 表示误差，初始设置为 $dx - dy$
2. 将 step 变成 $+sx, +sy$ ，分别表示 dx 与 dy 的符号

参考了 wikipedia Bresenham's line algorithm 相关内容。

实现伪代码如下（来源：wikipedia）

```
plotLine(x0, y0, x1, y1)
    dx = abs(x1 - x0)
    sx = x0 < x1 ? 1 : -1
    dy = -abs(y1 - y0)
```

```

sy = y0 < y1 ? 1 : -1
error = dx + dy

while true
  plot(x0, y0)
  if x0 == x1 && y0 == y1 break
  e2 = 2 * error
  if e2 >= dy
    error = error + dy
    x0 = x0 + sx
  end if
  if e2 <= dx
    error = error + dx
    y0 = y0 + sy
  end if
end while

```

结果如下

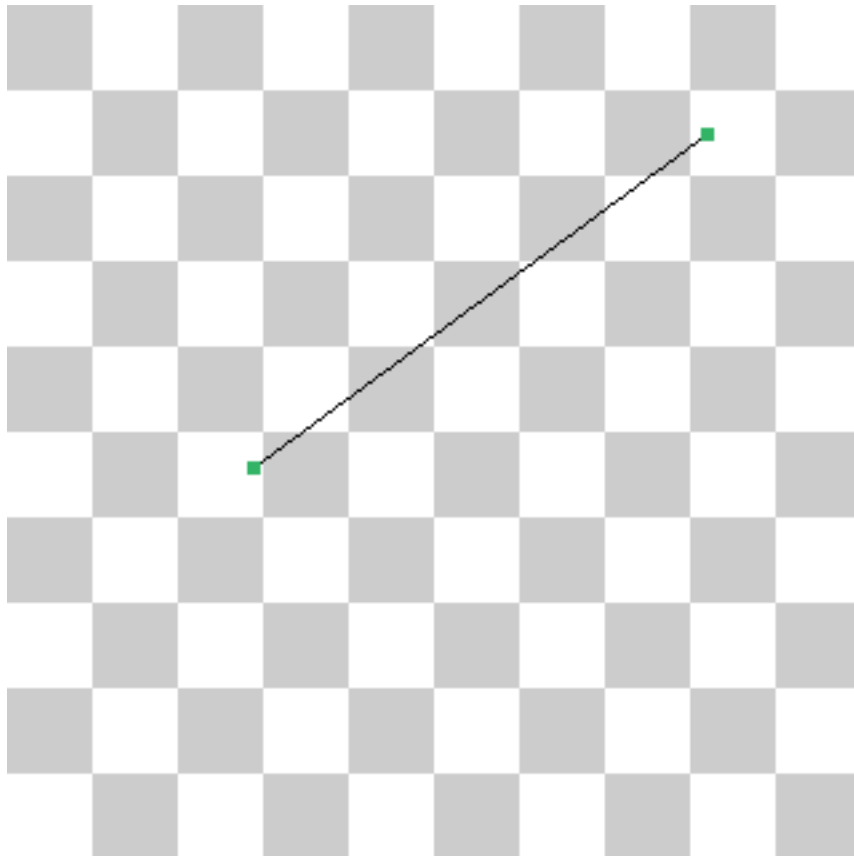


Figure 8: $k < 0$ 情况下画线

Task 5: Triangle Drawing (bonus=1')

先分析一下这个题目的基本框架。

我们可以先画出无填充的三角形，然后给他填充。

有些抽象，换言之，我们依次枚举每一行两条线的位置，将这两条线之间的点全部画出来，就画出了三角形。

为了实现这一点，我们首先要依据 y 对三个顶点排序。取 y 最小的点作为第一个顶点，取 y 次大的点作为第二个顶点，取 y 最大的点作为第三个顶点。

依次枚举 y , 计算出其对应的两个 x , 将这两个 x 之间的点全部染色, 我们就完成了这个程序的主体部分。

然而, 由于我们计算时会出现形如 $1/(y_2 - y_1)$ 的式子, 由于直线斜率可能为 0, 这就会带来很严重的数值不稳定性, 甚至造成 run time error。经过观察, 这个式子只有在 $y \neq y_2$ 的情况下才会被计算, 于是我们只需要对 $y = y_2$ 即枚举到第二个点的 y 值特殊处理即可。

结果如下

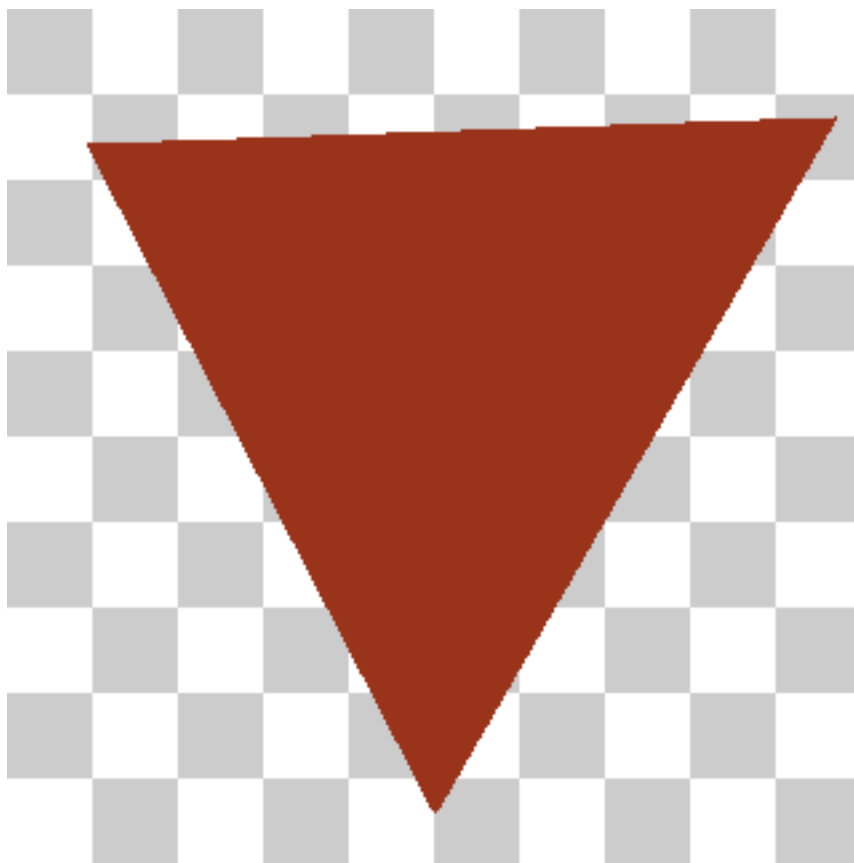


Figure 9: 一般情况下画三角形

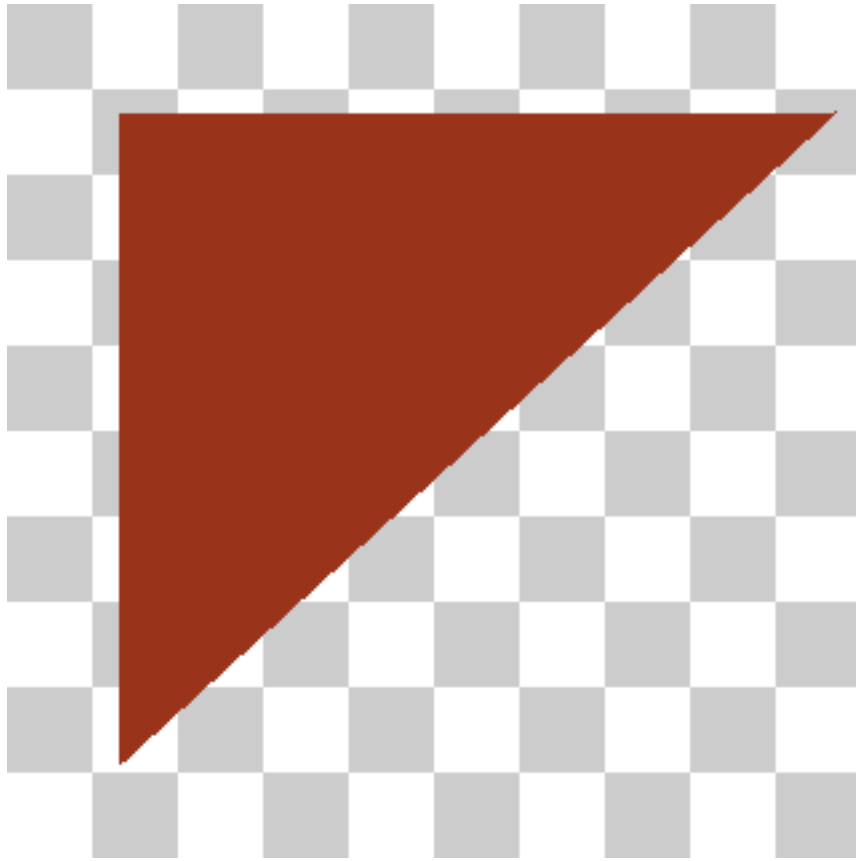


Figure 10: $k=0$ 下画三角形

Task 6: Image Supersampling (1')

依题意模拟即可。

先做一个下采样，求出对应 $\text{size} \times \text{size}$ 窗口下的平均值，将这个窗口的所有像素都设置为这个均值。

新建一个 320×320 的图像，找到这个新图上所有点对应的原位置，做双线性插值，就得到了结果。



Figure 11: Image Supersampling,rate=5

Task 7: Bezier Curve (1')

这道题给一个点集和一个 t , 只要求求出对应的 Bezier 曲线上的点即可。

使用 de Casteljau's algorithm 算法，递归求解。结果如下

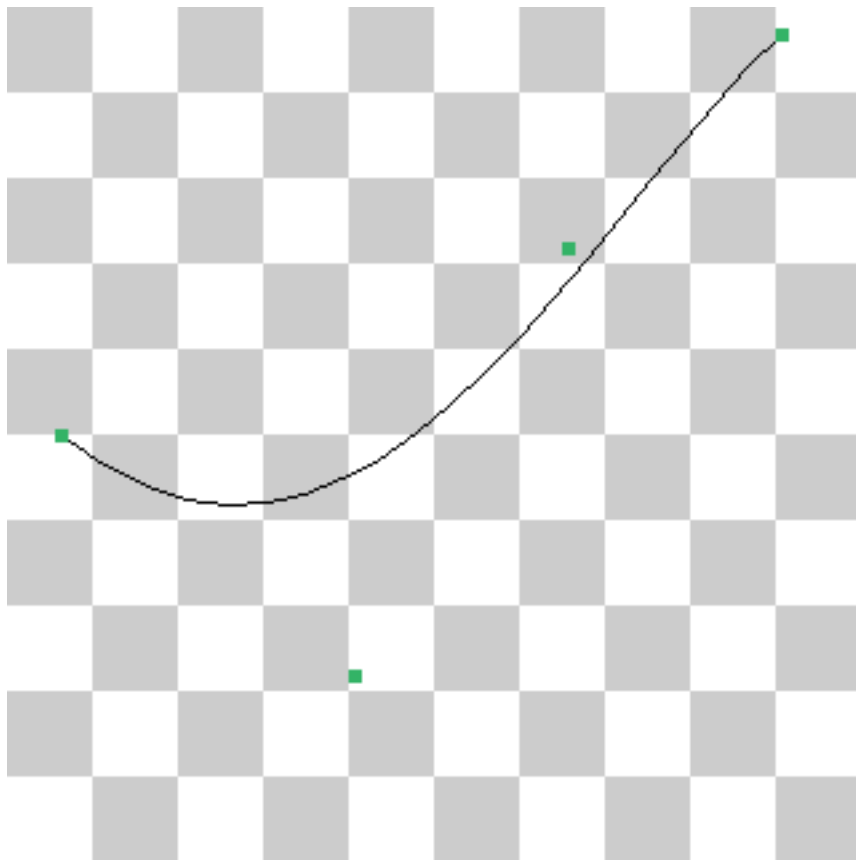


Figure 12: Bezier Curve

图像和代码可用性

报告中所有图片均在 `pic` 文件夹中，并附加了 `tasks.cpp` 文件，其包含了上述所有内容的实现代码。部分代码已经被注释。